Diseño de Sistemas de información



¿Qué es un Patrón de Diseño?

- Un patrón de diseño es una solución a un problema de diseño

- Se debe haber comprobado su efectividad resolviendo problemas similares en ocasiones anteriores.

- Debe ser reutilizable, lo que significa que es aplicable a diferentes problemas de diseño en distintas circunstancias

¿Para qué?

- Proporcionar elementos reusables en el diseño de sistemas software.
- Evitar la reiteración en la búsqueda de soluciones a problemas ya conocidos y solucionados anteriormente.
- Formalizar un vocabulario común entre diseñadores
- Estandarizar el modo en que se realiza el diseño.
- Facilitar el aprendizaje de las nuevas generaciones de diseñadores condensando conocimiento ya existente.

Que no buscan

- Imponer ciertas alternativas de diseño frente a otras.

- Eliminar la creatividad inherente al proceso de diseño.

- Ser de uso obligatorio, solo es aconsejable su uso en el caso de tener el mismo problema o similar que soluciona el patrón, siempre teniendo en cuenta que en un caso particular puede no ser aplicable. "Abusar o forzar el uso de los patrones puede ser un error"

Categorías de Patrones

- Patrones de arquitectura: Aquellos que expresan un esquema organizativo estructural fundamental para sistemas de software.
- Patrones de diseño: Aquellos que expresan esquemas para definir estructuras de diseño (o sus relaciones) con las que construir sistemas de software.
- Dialectos: Patrones de bajo nivel específicos para un lenguaje de programación o entorno concreto.

Clasificación del Patrón

- Patrones de creación: problemas relacionados a la creación de objetos.

- Patrones Estructurales: problemas relacionados con la composición de clases y objetos.

- Patrones de Comportamiento: problemas relacionados a la forma en la cual clases y objetos interactúan y distribuyen responsabilidades.

Ejercicio 1: enunciado

Una cadena de comida rápida necesita un sistema para gestionar el ciclo de vida de sus pedidos. Cada pedido puede pasar por los siguientes estados:

RECIBIDO → EN PREPARACIÓN → LISTO → ENTREGADO

Y también puede ser CANCELADO desde ciertos estados.

Se pide implementar una clase Pedido que permita:

1. Avanzar el pedido al siguiente estado con el método avanzarEstado()
2. Cancelar el pedido con el método cancelar()
3. Consultar el estado actual con obtenerEstado()
4. Cada operación debe imprimir un mensaje descriptivo de lo que ocurre

Restricciones de negocio:

- Un pedido **ENTREGADO** no puede avanzar ni cancelarse
- Un pedido **CANCELADO** no puede avanzar ni volver a cancelarse
- Un pedido **LISTO** no puede cancelarse (ya fue preparado)

Ejercicio 1: resolución

```java
public class Pedido {

    private int id;
    private String estado; // "RECIBIDO", "EN_PREPARACION", "LISTO", "ENTREGADO", "CAN

    public Pedido(int id) {
        this.id = id;
        this.estado = "RECIBIDO";
        System.out.println("Pedido #" + id + " creado. Estado: RECIBIDO");
    }
```

```java
public void cancelar() {
    if (estado.equals("RECIBIDO") || estado.equals("EN_PREPARACION")) {
        estado = "CANCELADO";
        System.out.println("Pedido #" + id + ": CANCELADO.");
    } else if (estado.equals("LISTO")) {
        System.out.println("❌ No se puede cancelar, el pedido ya está listo.");
    } else if (estado.equals("ENTREGADO")) {
        System.out.println("❌ No se puede cancelar, el pedido ya fue entregado.")
    } else if (estado.equals("CANCELADO")) {
        System.out.println("❌ El pedido ya estaba cancelado.");
    }
}
```

Ejercicio 1: resolución

```java
public void avanzarEstado() {
    if (estado.equals("RECIBIDO")) {
        estado = "EN_PREPARACION";
        System.out.println("Pedido #" + id + ": EN PREPARACIÓN");
    } else if (estado.equals("EN_PREPARACION")) {
        estado = "LISTO";
        System.out.println("Pedido #" + id + ": LISTO para retirar");
    } else if (estado.equals("LISTO")) {
        estado = "ENTREGADO";
        System.out.println("Pedido #" + id + ": ENTREGADO. ¡Buen provecho!");
    } else if (estado.equals("ENTREGADO")) {
        System.out.println("❌ El pedido ya fue entregado.");
    } else if (estado.equals("CANCELADO")) {
        System.out.println("❌ El pedido está cancelado.");
    }
}
```

```java
public String obtenerEstado() {
    return estado;
}
```

Ejercicio 1: modificación

"¿Qué pasa si mañana el negocio pide agregar un estado EN_CAMINO entre LISTO y ENTREGADO?"

Ejercicio 1: problemas de la solución planteada

Problema	Explicación
🔀 **Lógica dispersa**	Las reglas de cada estado están repartidas entre `avanzarEstado()` y `cancelar()`. Para entender qué puede hacer el estado LISTO, hay que leer *todos* los métodos.
📈 **Crecimiento cuadrático**	Cada nuevo estado obliga a tocar *todos* los métodos existentes. Con 3 métodos y 6 estados son 18 bloques a revisar.
🧨 **Alto riesgo de bugs**	Es fácil olvidar manejar un estado en algún método. El compilador no te avisa.
🔒 **String como contrato**	El estado es un `String` libre. Nadie impide escribir `"RECIBIDOO"` por error.
🧪 **Difícil de testear**	No se puede testear el comportamiento de un estado de forma aislada.
🚫 **Viola el principio Open/Closed**	Para agregar un estado hay que *modificar* código existente en lugar de *extenderlo*.

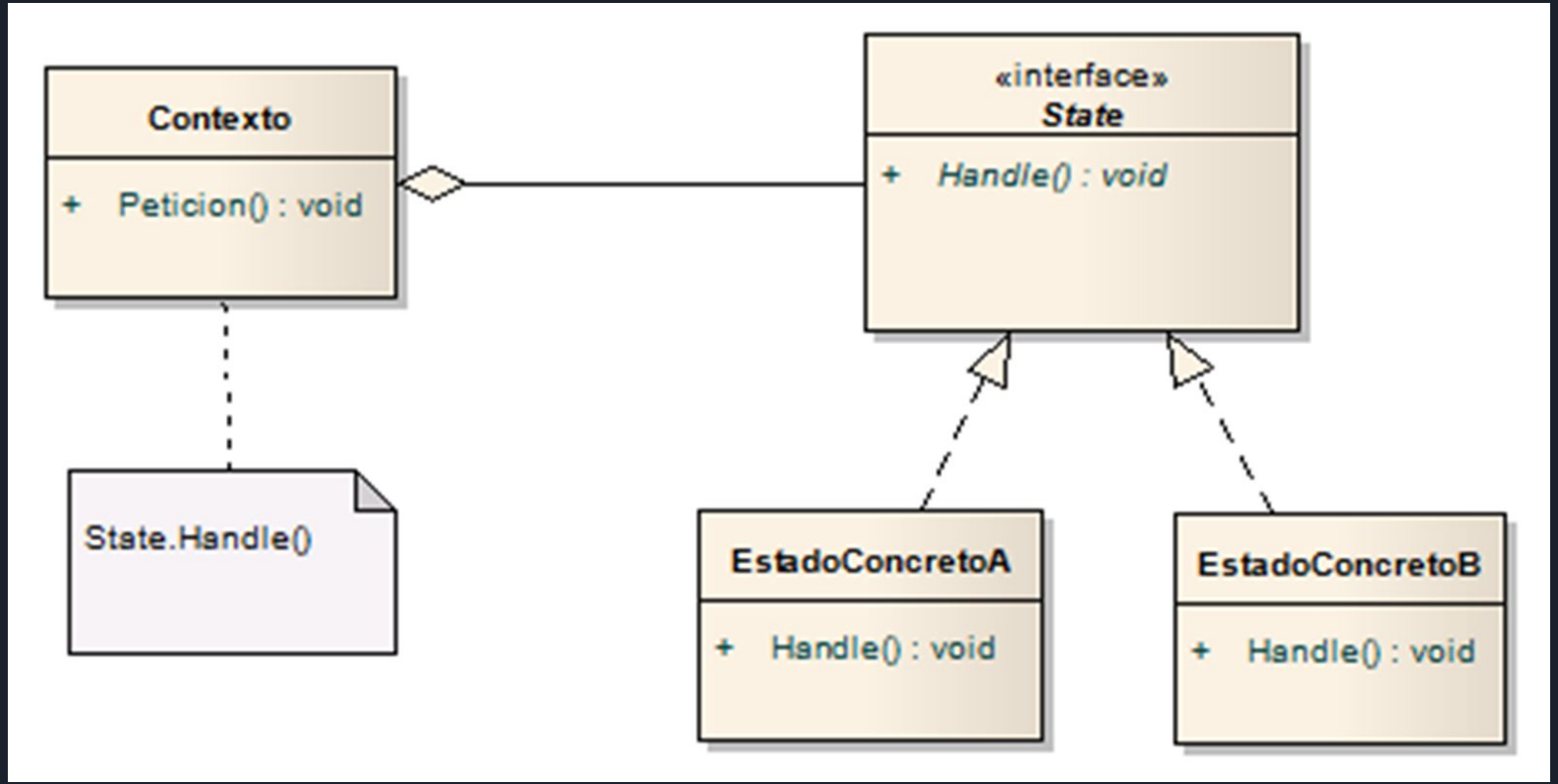
¿Cómo resolvemos el ejercicio 1 de forma más eficiente?

- NOMBRE DEL PATRÓN: State
- CLASIFICACIÓN DEL PATRÓN: De comportamiento
- INTENCIÓN: Permitir que un objeto modifique su comportamiento cuando su estado cambie. Parecerá que el objeto cambia de clase.
- MOTIVACIÓN: El patrón State está motivado por aquellos objetos en que varía su comportamiento ante los diferentes mensajes según su estado actual.

STATE

- APLICABILIDAD:
  - en entornos en los que se identifiquen “estados” con operaciones comunes que varían su comportamiento dependiendo de en cuál de ellos se encuentra.
  - El código que varía, en cada operación es mucho.

STATE - ESTRUCTURA



Ejercicio 1: resolución con patrón STATE

```java
// 1. Interfaz
public interface EstadoPedido {
    void avanzarEstado(Pedido pedido);
    void cancelar(Pedido pedido);
    String obtenerEstado();  // ← corregido
}
```

Ejercicio 1: resolución con patrón STATE

```java
// 2. Estados
public class EstadoRecibido implements EstadoPedido {
    public void avanzarEstado(Pedido pedido) {
        System.out.println("✅ Pedido en PREPARACIÓN.");
        pedido.setEstado(new EstadoEnPreparacion());
    }
    public void cancelar(Pedido pedido) {
        System.out.println("✅ Pedido CANCELADO.");
        pedido.setEstado(new EstadoCancelado());
    }
    public String obtenerEstado() { return "RECIBIDO"; }  // ← c
}

public class EstadoEnPreparacion implements EstadoPedido {
    public void avanzarEstado(Pedido pedido) {
        System.out.println("✅ Pedido LISTO para retirar.");
        pedido.setEstado(new EstadoListo());
    }
    public void cancelar(Pedido pedido) {
        System.out.println("✅ Pedido CANCELADO.");
        pedido.setEstado(new EstadoCancelado());
    }
    public String obtenerEstado() { return "EN PREPARACIÓN"; }
}
```

Ejercicio 1: resolución con patrón STATE

```
public class EstadoListo implements EstadoPedido {
    public void avanzarEstado(Pedido pedido) {
        System.out.println("✅ Pedido ENTREGADO. ¡Buen provecho!");
        pedido.setEstado(new EstadoEntregado());
    }
    public void cancelar(Pedido pedido) {
        System.out.println("❌ No se puede cancelar, el pedido ya está listo.");
    }
    public String obtenerEstado() { return "LISTO"; }  // ← corregido
}

public class EstadoEntregado implements EstadoPedido {
    public void avanzarEstado(Pedido pedido) {
        System.out.println("❌ El pedido ya fue entregado.");
    }
    public void cancelar(Pedido pedido) {
        System.out.println("❌ El pedido ya fue entregado.");
    }
    public String obtenerEstado() { return "ENTREGADO"; }  // ← corregido
}
```

Ejercicio 1: resolución con patrón STATE

```java
public class EstadoCancelado implements EstadoPedido {
    public void avanzarEstado(Pedido pedido) {
        System.out.println("❌ El pedido está cancelado.");
    }
    public void cancelar(Pedido pedido) {
        System.out.println("❌ El pedido ya estaba cancelado.");
    }
    public String obtenerEstado() { return "CANCELADO"; }  // ← cor
}
```

Ejercicio 1: resolución con patrón STATE

```java
// 3. Contexto
public class Pedido {
    private int id;
    private EstadoPedido estado;

    public Pedido(int id) {
        this.id = id;
        this.estado = new EstadoRecibido();
        System.out.println("Pedido #" + id + " creado. Estado: " + estado.obtenerEstac
    }

    public void avanzarEstado()  { estado.avanzarEstado(this); }
    public void cancelar()       { estado.cancelar(this); }
    public void setEstado(EstadoPedido nuevoEstado) { this.estado = nuevoEstado; }
    public String obtenerEstado() { return estado.obtenerEstado(); }  // ← delega al e
}
```

Catálogo de patrones

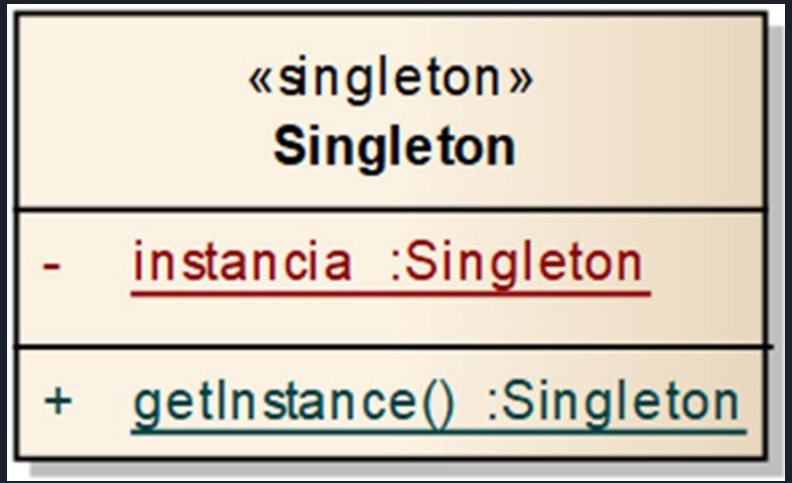
		Propósito		
		de creación	de estructura	de comportamiento
Alcance	A nivel de clase	·Factory Method	·Adapter	·Interpreter ·Template Method
	A nivel de objeto	·Abstract Factory ·Builder ·Prototype ·Singleton	·Adapter ·Bridge ·Composite ·Decorator ·Facade ·Proxy	·Chain of Responsibility ·Command ·Iterator ·Mediator ·Memento ·Flyweight ·Observer ·State ·Strategy ·Visitor

SINGLETON

- NOMBRE DEL PATRÓN: Singleton.
- CLASIFICACIÓN DEL PATRÓN: De creación de Objetos
- INTENCIÓN: Asegurar que una clase tenga una única instancia y proporcionar un punto de acceso global
- MOTIVACIÓN: Muchas clases tienen la necesidad de tener una única instancia (por ejemplo: cuando una clase controla un recurso físico único) o cuando cierto tipo de datos debe estar disponible para todos los demás objetos de la aplicación.

SINGLETON

- APLICABILIDAD:
  - Debe existir una única instancia de una clase, y el acceso a la misma debe ser un punto bien conocido.



SINGLETON

```
package com.miejemplo;


public class Configurador {


private String url;
 private String baseDatos;
 private static Configurador miconfigurador;
//
```

SINGLETON

```
//
public  syncronized static Configurador getInstance( ) {
        if (miconfigurador==null) {
                //Lee los url y base de datos de un archivo de configuración
                miconfigurador=new Configurador(url,baseDatos);
        }
        return miconfigurador;
}
//
```

SINGLETON

```
//
 private Configurador(String url,String baseDatos){

        this.url=url;
        this.baseDatos=baseDatos;

 }
//
```

SINGLETON

```
//
 public String getUrl() { return url; }

public void setUrl(String url) { this.url = url; }

public String getBaseDatos() { return baseDatos; }

public void setBaseDatos(String baseDatos) { this.baseDatos =
baseDatos;}
}
```

SINGLETON - Cliente

```
package com.miejemplo;
public class Principal {
public static void main(String[] args) {

        Configurador c= Configurador.getInstance( );

        System.out.println(c.getUrl());
        System.out.println(c.getBaseDatos());
}
```

SINGLETON - CONSIDERACIONES

- El objeto no debe ser clonable.

```
public Object clone() throws CloneNotSupportedException {
    throw new CloneNotSupportedException;
}
```

- Los métodos o la clase deben ser declarado como final.